

Vasic-Digital-Reusable-Module-Suite

Construisez une fois, réutilisez partout — une flotte de modules Go et KMP petits, découplés et testés indépendamment.

Résumé

Une vaste famille de modules génériques et réutilisables, publiés sous les espaces de noms `digital.vasic.*` (Go) et Kotlin Multiplatform. Chaque module est autonome, testé et versionné de manière indépendante, et intégré comme sous-module à code égal dans des produits plus larges (Catalogizer, HelixAgent et l'ensemble de la flotte). Cette page regroupe les nombreuses petites utilités qui, prises individuellement, créeraient du bruit en tant que pages distinctes.

Description courte

Une suite soigneusement sélectionnée de modules découplés `digital.vasic.*` — des primitives d'infrastructure (authentification, cache, base de données, configuration, observabilité), des briques de construction AI/agent (RAG, VectorDB, Embeddings, MCP, Agentic, Planification), et des garde-fous défensifs-LLM (RedTeam, Normalize) — complétée par un jeu miroir Kotlin Multiplatform. Chacun est générique, testé et réutilisable.

Description longue

L'organisation vasic-digital repose sur un pari structurel : une philosophie de « constitution + nombreux sous-modules réutilisables découplés », où aucune fonctionnalité générique n'est écrite deux fois. Plutôt que des monolithes, chaque préoccupation réutilisable est extraite dans son propre petit module — son propre dépôt, ses propres tests, sa propre documentation — et maintenue strictement découplée pour éviter toute fuite des spécificités d'un consommateur. Cette page les

regroupe car, pris un à un, chacun relève de l'échelle d'une bibliothèque et serait du bruit en tant que page produit individuelle. Ensemble, ils constituent le véritable multiplicateur de force de l'organisation : un actif d'ingénierie privé qui transforme « construire un nouveau produit » en « assembler des composants éprouvés », et la concrétisation de l'affirmation selon laquelle cette flotte ne réinvente pas la roue — elle entretient une roue très performante et la déploie partout.

La suite se décline en trois ensembles. **Les primitives d'infrastructure** (Go) fournissent les fondations nécessaires à chaque service : auth (JWT/bcrypt), cache (Redis/TTL), database (migrations, double SQLite/PostgreSQL), config, middleware, observabilité (Prometheus/OpenTelemetry), ratelimiter, sécurité, stockage (S3/MinIO), streaming (hub WebSocket), eventbus, système de fichiers (multi-protocole), découverte/mdns, http3, recovery, concurrency, lazy, et bien d'autres. **Les briques de construction AI/agent** (Go) offrent le substrat des systèmes AI : rag, vectordb, embeddings, mémoire, conversation (compression de contexte infini, event sourcing), mcp (Model Context Protocol), toolschema, skillregistry, agentic (orchestration de workflows basée sur des graphes), planning (HiPlan/MCTS/Tree-of-Thoughts), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (modélisation de récompenses/RLHF), et toon (Token-Oriented Object Notation). **Les garde-fous défensifs-LLM** fournissent des outils de robustesse adversariale : RedTeam (fixtures adversariales pilotées par YAML), Normalize (canonisation d'entrées adversariales). Un ensemble **Kotlin Multiplatform** parallèle reflète les modules principaux (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP, etc.) pour les applications multiplateformes.

Pourquoi nous l'avons conçu

Développer à chaque fois de nombreux produits (Catalogizer, HelixAgent, Herald et autres) à partir de zéro est à la fois inefficace et incohérent. En extrayant chaque préoccupation générique dans un module découplé et testé, les corrections et améliorations se propagent à l'ensemble du parc, et chaque nouveau produit s'assemble à partir de composants éprouvés.

Pourquoi c'est révolutionnaire

Il s'agit, en pratique, d'une « bibliothèque standard » privée pour la construction de backends centrés sur AI — cette couche que la plupart des équipes n'ont jamais l'occasion de développer, trop occupées à réinventer l'authentification, la mise en cache ou les fondations RAG pour la cinquième fois. Ici, les primitives d'infrastructure, les briques de construction AI et les garde-fous défensifs LLM existent sous forme de modules autonomes et

testés indépendamment. C'est ce qui permet à une petite équipe de livrer des systèmes de qualité production à un rythme normalement réservé à des effectifs bien plus importants, et ce, sans accumuler la dette de duplication qui accompagne habituellement ce type de développement.

Ce qui est innovant

- Une discipline de découplage à l'échelle du parc (CONST-051) : les sous-modules sont traités comme des bases de code à part entière, sans intégrer de spécificités consommateur.
- Une couche dédiée aux primitives AI (RAG, VectorDB, Embeddings, MCP, ToolSchema, Agentic, Planning, LLMOps) sous forme de modules réutilisables.
- Un cluster de garde-fous défensifs LLM (RedTeam, Normalize) pour une robustesse face aux attaques.
- Des ensembles de modules Go et Kotlin Multiplatform parallèles partageant les mêmes conventions.

Défis et solutions

- **Éviter la dégradation des couplages entre des dizaines de modules** : résolu par le contrat de découplage de la constitution et l'injection à l'exécution des spécificités consommateur.
- **Maintenir la cohérence et les tests sur de nombreux modules** : résolu par une convention commune (tests/docs/ Défis par module) et l'épine dorsale de gouvernance HelixConstitution.
- **Portée multiplateforme** : résolu par un miroir Kotlin Multiplatform des modules principaux.

Pile technologique (pourquoi et comment)

- **Go** — la majorité des modules (digital.vasic.*).
- **Kotlin Multiplatform** — modules miroirs multiplateformes (Auth/Database/Sécurité/UI/Concurrency/RateLimiter-KMP).
- **Redis / PostgreSQL / SQLite** — primitives de cache, base de données et stockage.
- **Prometheus / OpenTelemetry** — module d'observabilité.
- **WebSocket / HTTP/3 (quic-go) / mDNS** — modules réseau.
- **Vector DB / embeddings / RAG / MCP** — modules de primitives AI.
- **YAML** — fixtures et configuration adversariales RedTeam.

NON VÉRIFIÉ / EN COURS : plusieurs dépôts de l'organisation sont marqués « ÉCHAFAUDAGE / EN COURS » (ex. : PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions). À présenter comme des projets en phase initiale/échaudage, non livrés.